Software Design

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

System Analysis

- It is a process of collecting and interpreting facts, finding and gathering requirements, identifying the problems, and decomposition of a system into its components.
- Analysis specifies **what the system should do.**
- System analysis is conducted for the purpose of studying a system or its parts in order to identify its objectives.
- It is a problem solving technique that improves the system and ensures that all the components of the system work efficiently to accomplish their purpose.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

System Analyst

- System analyst is a person who is thoroughly aware of the system and guides the system development project by giving proper directions.
- Is an expert having technical and interpersonal skills to carry out development tasks required at each phase.
- Main Roles

- Defining and understanding the requirement of user through various Fact finding techniques.
- Prioritizing the requirements by obtaining user consensus.
- Gathering the facts or information and acquires the opinions of users.
- Suggest many flexible alternative solutions, pick the best solution, and quantify cost and benefits.
- Draw certain specifications which are easily understood by users and programmer in precise and detailed form.
- Implemented the logical design of system which must be modular.
- Plan the periodicity for evaluation after it has been used for some time, and modify the system as needed

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

System Design

- Systems design is the process of defining the architecture, modules, interfaces, and data for a system to satisfy specified requirements.
- **Describes how the analyzed requirements can be met**.
- System design is the process of designing the elements of a system such as the architecture, modules and components, the different interfaces of those components and the data that goes through that system.
- Software system design is a process through which the requirements are translated into a representation of software
- It is the process of defining, developing and designing systems which satisfies the specific needs and requirements of a business or organization.
- The purpose of the System Design process is to provide sufficient detailed data and information about the system and its system elements to enable the implementation consistent with architectural entities as defined in models and views of the system architecture.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

System Design

- A systemic approach is required for a coherent and well-running system.
- Bottom-Up or Top-Down approach is required to take into account all related variables of the system.
- Tells the customer what the system will do
  - Where will the data come from?
  - What will happen to the data in the system?
  - What will the system look like to users?
  - What choices will be offered to users?
  - What is the timing of events?
  - What will the reports and screens look like?
- Tells the programmers what the system will do
  - Major hardware components and their function
  - Hierarchy and functions of software components
  - Data structures
  - Data flow

Available on notes.arpanadhikari7.com.np

System Design

➢Major elements of a System are:

- **Architecture -** This is the conceptual model that defines the structure, behavior and more views of a system. We can use flowcharts to represent and illustrate the architecture.
- **Modules -** This are components that handle one specific tasks in a system. A combination of the modules make up the system.
- **Components -** This provides a particular function or group of related functions. They are made up of modules.
- **Interfaces -** This is the shared boundary across which the components of a the system exchange information and relate.
- **Data -** This the management of the information and data flow.

Available on notes.arpanadhikari7.com.np

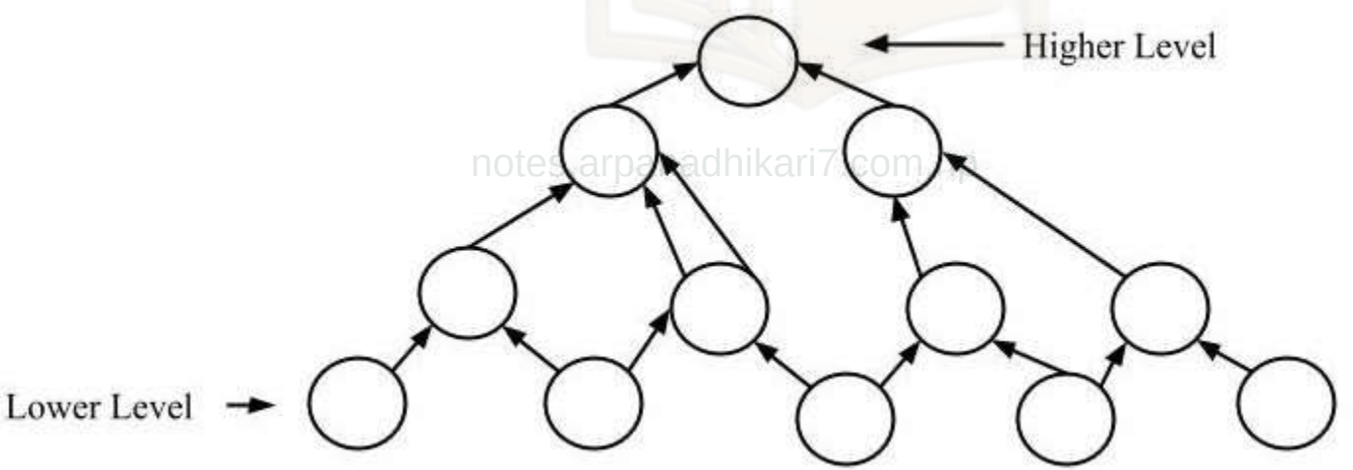
Strategies For Performing System Design

1. Bottom-Up Approach
2. Top-Down Approach
3. Hybrid Design

notes.arpanadhikari7.com.np

Strategies For Performing System Design-Bottom-Up Approach

- The design starts with the lowest level components and subsystems.
- By using these components, the next immediate higher level components and subsystems are created or composed.
- The process is continued till all the components and subsystems are composed into a single component, which is considered as the complete system.
- The amount of abstraction grows high as the design moves to more high levels.



Available on notes.arpanadhikari7.com.np

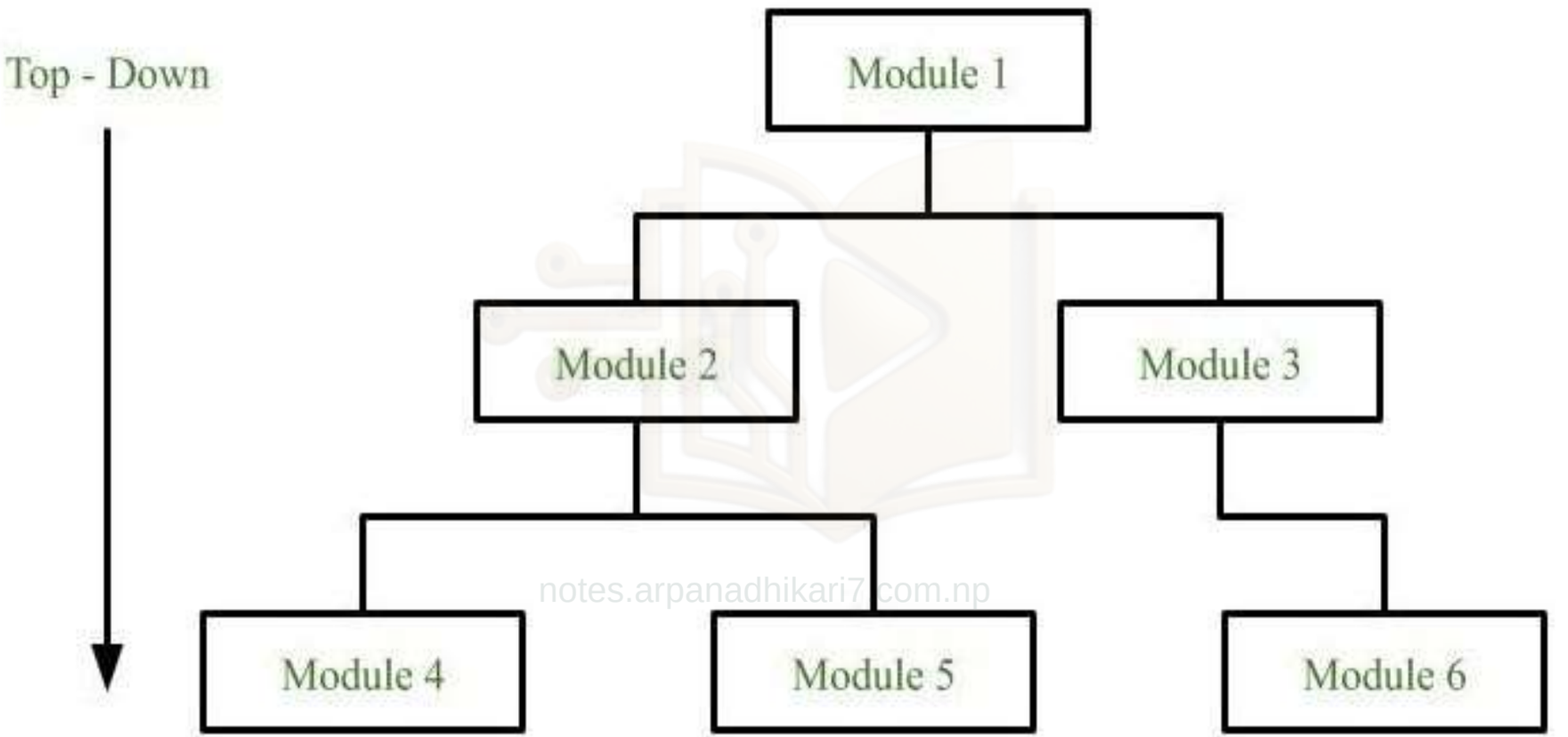
Strategies For Performing System Design-Top-Down Approach

- Each system is divided into several subsystems and components.
- Each of the subsystem is further divided into set of subsystems and components.
- This process of division facilitates in forming a system hierarchy structure.
- The complete software system is considered as a single entity and in relation to the characteristics, the system is split into sub-system and component.
- The same is done with each of the sub-system.
- This process is continued until the lowest level of the system is reached.
- The design is started initially by defining the system as a whole and then keeps on adding definitions of the subsystems and components.
- When all the definitions are combined together, it turns out to be a complete system.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Strategies For Performing System Design-Top-Down Approach



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Strategies For Performing System Design-Hybrid Design

- It is a combination of both the top–down and bottom–up design strategies.
- In this we can reuse the modules.

notes.arpanadhikari7.com.np

Inputs to System Design

- Statement of work
- Requirement determination plan
- Current situation analysis
- Proposed system requirements including a conceptual data model, modified DFDs, and Metadata (data about data).

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Outputs for System Design

- Infrastructure and organizational changes for the proposed system.
- A data schema, often a relational schema.
- Metadata to define the tables/files and columns/data-items.
- A function hierarchy diagram or web page map that graphically describes the program structure.
- Actual or pseudo code for each module in the program.
- A prototype for the proposed system

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Types/Methods of System Design

1. Logical Design
2. Physical Design
3. Architectural Design
4. Detailed Design

notes.arpanadhikari7.com.np

Types/Methods of System Design-Logical Design

- A logical design is a conceptual, abstract design.
- We do not deal with the physical implementation details.
- Logical design is a graphical representation of a system showing the system's processes and the flows of data into and out of the process.
- The process of logical design involves arranging data into a series of logical relationships called entities and attributes.
- It describes the inputs (sources), outputs (destinations), databases (data stores), procedures (data flows) all in a format that meets the user requirements.
- The logical design process helps us to implement a database that satisfies the requirements of our business organization.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Types/Methods of System Design-Physical Design

- Physical design relates to the actual input and output processes of the system.
- It basically shows the system's internal and external entities and the flow of data into and out of these entities.
- It focuses on how data is entered into a system, verified, processed, and displayed as output.
- It is concerned with user **interface design, process design** and **data design.**
- It consists of the following steps −
  - Specifying the input/output media, designing the database, and specifying backup procedures.
  - Planning system implementation.
  - Devising a test and implementation plan, and specifying any new hardware and software.
  - Updating costs, benefits, conversion dates, and system constraints

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Types/Methods of System Design-Architectural Design

- It is also known as high level design that focuses on the design of system architecture.
- Architectural Design is used to describes the views, models, behavior, and structure of the system.
- It defines the structure and relationship between various modules of system development process.
- In architectural design, the overall structure of the system is chosen, but the internal details of major components are ignored.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Types/Methods of System Design-Detailed Design

- Detailed design deals with the implementation part
- It follows Architectural design and focuses on development of each module.
- The detailed design may include:
  - Decomposition of major system components into program units.
  - Allocation of functional responsibilities to units.
  - User interfaces
  - Unit states and state changes
  - Data and control interaction between units
  - Data packaging and implementation, including issues of scope and visibility of program elements
  - Algorithms and data structures

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Factors that Affect Technology Trade-offs during System Design

1. Scale of Product
2. Time: Learning new technologies sometimes often takes time. The trade-offs in this instance will be made according to which stack/technology will be in time with the set delivery dates.
3. Cost
4. Efficiency: Technology trade offs are also done based on which technology is more efficient
5. User Experience and Support
6. Maintainability
7. Reliability
8. Scalability

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Software Design

- Software design is an iterative process through which requirements are translated into the blueprint for building the software.
- The design process comprises a set of principles, concepts and practices, which allow a software engineer to model the system or product that is to be built.
- Once the requirements document for the software to be developed is available, the software design phase begins.
- While the requirement specification activity deals entirely with the problem domain, design is the first phase of transforming the problem into a solution.
- The software design process can be divided into the following three levels of phases of design:
  - Interface Design
  - Architectural Design
  - Detailed Design

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Software Design-Interface Design

- Interface design is the specification of the interaction between a system and its environment.
- During interface design, the internal of the systems are completely ignored and the system is treated as a black box.
- Interface design should include the following details:
  - Precise description of events in the environment, or messages from agents to which the system must respond.
  - Precise description of the events or messages that the system must produce.
  - Specification on the data, and the formats of the data coming into and going out of the system.
  - Specification of the ordering and timing relationships between incoming events or messages, and outgoing events or outputs.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Software Design-Architectural Design

- It is also known as high level design that focuses on the design of system architecture.
- Architectural Design is used to describes the views, models, behavior, and structure of the system.
- It defines the structure and relationship between various modules of system development process.
- In architectural design, the overall structure of the system is chosen, but the internal details of major components are ignored.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Software Design-Detailed Design

- Detailed design deals with the implementation part
- It follows Architectural design and focuses on development of each module.
- The detailed design may include:
  - Decomposition of major system components into program units.
  - Allocation of functional responsibilities to units.
  - User interfaces
  - Unit states and state changes
  - Data and control interaction between units
  - Data packaging and implementation, including issues of scope and visibility of program elements
  - Algorithms and data structures

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Software Design Concepts

1. Abstraction
2. Architecture
3. Patterns
4. Modularity
5. Information Hiding
6. Stepwise Refinement
7. Refactoring
8. Concurrency

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Software Design Concepts-Abstraction

- Abstraction is one of the key concepts in software design and development.
- Its main goal is to handle complexity by hiding unnecessary details from the user.
- Abstraction is the act of representing essential features without including the background details or explanations.
- Allows software designers to consider components at an abstract level, while neglecting the implementation details of the components.
- In the computer science and software engineering domain, the abstraction principle is used to reduce complexity and allow efficient design and implementation of complex software systems.
- **Real World Example:** Making coffee with a coffee machine is a good example of abstraction. You need to know how to use your coffee machine to make coffee. The thing you don't need to know is how the coffee machine is working internally to brew a fresh cup of delicious coffee. You don't need to know the ideal temperature of the water or the amount of ground coffee you need to use.

Available on notes.arpanadhikari7.com.np

Software Design Concepts-Abstraction

- There are three commonly used abstraction mechanisms in software design, namely, functional abstraction, data abstraction and control abstraction.
- All these mechanisms allow us to control the complexity of the design process by proceeding from the abstract design model to concrete design model in a systematic manner.

1. **Functional abstraction:** This involves the use of parameterized subprograms. Functional abstraction can be generalized as collections of subprograms referred to as ‘groups’.
2. **Data abstraction:** This involves specifying data that describes a data object.
3. **Control abstraction:** This states the desired effect, without stating the exact mechanism of control.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Software Design Concepts-Architecture

- Software architecture refers to the structure of the system, which is composed of various components.
- Software architecture comprises two elements of design model, namely, data design and architectural design.
- The software architecture enables the software engineers to analyze the software design efficiently.
- The software architecture does the following.
  - Enables the software engineers to analyze the software design efficiently.
  - Provides an insight to all the interested stakeholders that enable them to communicate with each other.
  - Highlights early design decisions, which have great impact on the software engineering activities (like coding and testing) that follow the design phase
  - Creates intellectual models of how the system is organized into components and how these components interact with each other.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Software Design Concepts-Pattern

- A pattern provides a description of the solution to a recurring design problem of some specific domain in such a way that the solution can be used again and again.
- Uses of design pattern:
  - Patterns are very useful in creating good software design solutions.
  - Patterns include a clear specification of the problem, and also explain the circumstances in which the solution would and would not work.
  - Patterns can speed up the development process by providing tested, proven development paradigms.
  - Patterns provide general solutions, documented in a format that doesn't require specifications tied to a particular problem.
  - Patterns allow developers to communicate using well-known, well understood names for software interactions.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Software Design Concepts-Pattern

➢Types of Design Patterns

1. **Architectural patterns:** These patterns are high-level strategies that refer to the overall structure and organization of a software system.
2. **Design patterns:** These patterns are medium-level strategies that are used to solve design problems. They provide a means for the refinement of the elements (as defined by architectural pattern) of a software system or the relationship among them.
3. **Idioms:** These patterns are low-level patterns, which are programming-language specific. They describe the implementation of a software component, the method used for interaction among software components, etc., in a specific programming language.

Available on notes.arpanadhikari7.com.np

Software Design Concepts-Modularity

- Modularity is achieved by dividing the software into uniquely named and addressable components, which are also known as modules.
- It is the process of decomposing system into loosely coupled modules(modules less dependent upon other).
- After developing the modules, they are integrated together to meet the software requirements.
- Larger the number of modules a system is divided into, greater will be the effort required to integrate the modules.
- **Coupling** is the degree of interdependence between software modules.
- **Cohesion** defined as the degree to which all elements of a module, class, or component work together as a functional unit.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Software Design Concepts-Information Hiding

- The way of hiding unnecessary details is referred to as information hiding.
- Modules should be specified and designed in such a way that the data structures and processing details of one module are not accessible to other modules.
- They pass only that much information to each other, which is required to accomplish the software functions.
- Some of the advantages associated with information hiding are listed below.
  - Leads to low coupling
  - Emphasizes communication through controlled interfaces
  - Decreases the probability of adverse effects
  - Restricts the effects of changes in one component on others
  - Results in higher quality software.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Software Design Concepts-Stepwise Refinement

- Stepwise refinement is a top-down design strategy used for decomposing a system from a high level of abstraction into a more detailed level (lower level) of abstraction.
- At the highest level of abstraction, function or information is defined conceptually without providing any information about the internal workings of the function or internal structure of the data.
- As we proceed towards the lower levels of abstraction, more and more details are available.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Software Design Concepts-Refactoring

- Refactoring can be defined as a process of modifying a software system to improve the internal structure of design without changing its external behavior.
- Refactoring is an important design activity that reduces the complexity of module design keeping its behavior or function unchanged.
- During the refactoring process, the existing design is checked for any type of flaws like redundancy, poorly constructed algorithms and data structures, etc., in order to improve the design.
- Software designers may decide to refactor the components, each exhibiting high cohesion

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Software Design Concepts-Concurrency

- Computer has limited resources and they must be utilized efficiently as much as possible.
- To utilize these resources efficiently, multiple tasks must be executed concurrently.
- This requirement makes concurrency one of the major concepts of software design.
- Every system must be designed to allow multiple processes to execute concurrently, whenever possible.
- However, concurrent execution of multiple processes sometimes may result in undesirable situations such as an inconsistent state, deadlock, etc.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Object Oriented Analysis(OOA)

- **Analysis e**mphasizes on investigation of the problem and requirements, rather than a solution.
- In object oriented analysis, the system requirements are determined, the concepts are identified and conceptual model is built.
- It focuses on finding or describing the concepts on the problem domain.
- In Object Oriented Analysis part we:
  - **Requirements Elicitation***:* Define what does the software need to do, and what's the problem the software trying to solve.
  - **Specify requirements**: Describe the requirements, usually, using use cases (and scenarios) or user stories.
  - **Build Conceptual model:** Identify the important objects, refine them, and define their relationships and behavior among them and draw them in a simple diagram.

  For example, in the case of library management system, some of the concepts are **Book**, **Library** and **Librarian**.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Object Oriented Design(OOD)

- During object-oriented design, the focus is on defining software objects and how they collaborate to fulfill the requirements.
- OOD transforms the analysis model created using OOA into a design model that serves as a blueprint for software construction.
- Object-oriented design is concerned with developing an object-oriented model of a software system to implement the identified requirements.
- After the analysis phase, the conceptual model is developed further into an object-oriented model using object-oriented design (OOD).
- In an object-oriented design a software system is designed as a set of interacting objects that manage their own private state and offer services to other objects.
- Implementation constraints such as concurrency, distribution, persistence are dealt during this phase.
- For example, in the case of library management system, a Book software object may have a **title** attribute and a **getChapter** method.

Available on notes.arpanadhikari7.com.np

How are OOA, OOD and OOP related?

- In OOA, the system requirements are determined, the objects are identified, the conceptual model is developed.
- After the analysis phase, the conceptual model is developed further into an object-oriented model using object-oriented design (OOD). The product of OOA serves as the model from which we may start an OOD.
- The product of OOD can then be used as blueprint for completely implementing a system using OOP methods. OOP is concerned with realising an OOD using an object-oriented programming language such as Java, C++,etc.
- Object-oriented analysis, design and programming are related but distinct. OOA is concerned with developing an object model of the application domain, OOD is concerned with developing an object-oriented system model to implement requirements and OOP is concerned with realising an OOD using an OO programming language.

Available on notes.arpanadhikari7.com.np

Object Oriented Software Development

- Object oriented systems development is a way to develop software by building self-contained modules or objects that can be easily replaced, modified and reused.
- Object-oriented development is highly incremental approach. We start with object-oriented analysis, model it, create an object-oriented design, then do some more of each, again and again, gradually refining and completing models of the system.
- The Object-Oriented approach of Building Systems takes the objects as the basis.
- Object oriented SDLC consists of three macro processes:

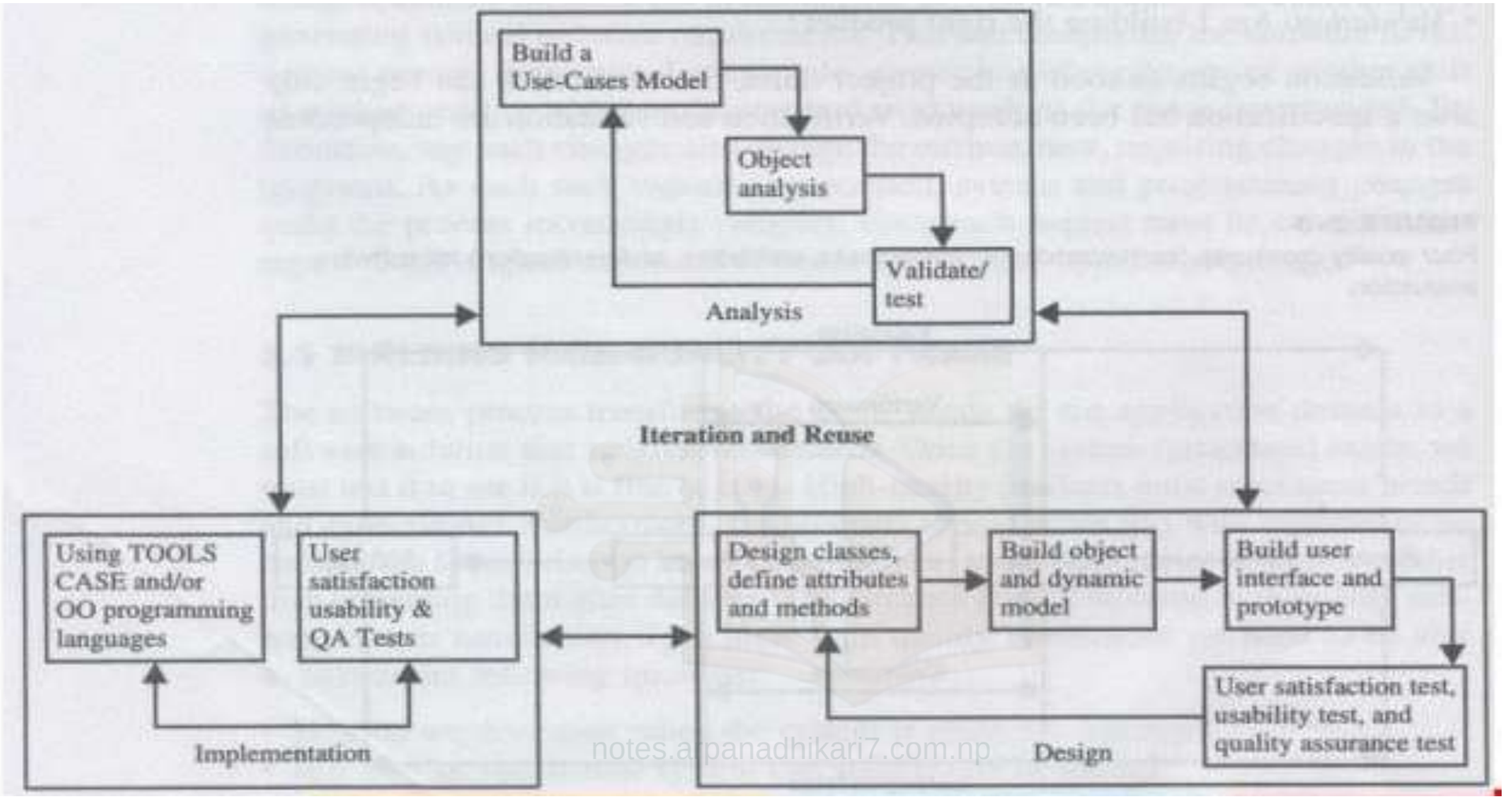
a. Object oriented Analysis- Use Case Driven

b. Object oriented Design

c. Object oriented Implementation

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Figure: Object Oriented System/Software Development Approach

Available on notes.arpanadhikari7.com.np

UML(Unified Modeling Language)

- The Unified Modeling Language(UML) is a visual language for specifying, constructing and documenting the artifacts(work product) of systems.
- Unified Modeling language (UML) is a standardized modeling language enabling developers to specify, visualize, construct and document artifacts of a software system.
- UML is an important aspect involved in object-oriented software development. It uses graphic notation to create visual models of software systems.
- UML is not a system design or development methodology, but can be used to document object-oriented and analysis results obtained using some methodology.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Why use UML?

- Permits us to specify the structure or behavior of a system.
- Helps us visualize and better understand the system that we are building.
- Provides system architects, software engineers, and software developers with tools for analysis, design, and implementation of software-based systems as well as for modeling business and similar processes.
- Provides template that guides us in constructing a system.
- Helps to understand complex system part by part.
- Document the decisions that we have made.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Ways of Applying UML

a. UML as sketch:

- Sketch is the most common use of the UML
- Sketches are informal diagrams used to communicate ideas, explore alternatives or design in a collaborative manner.
- They are usually focused on some aspect of the system and are not intended to show every detail of it.

b. UML as blueprint:

- The purpose is to describe a system in detail mostly used in forward and reverse engineering.
- In the case of forward engineering, the details of the blueprint should be enough for a programmer to code the system or generate the code.
- In case of reverse engineering UML Diagrams helps to visualize and better understand existing code.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

c. UML as a Programming Language

- UML diagrams can be used to generate code which can be further modified by developers.
- There is no any formal definition that exists of how UML maps to any particular programming language.

Agile Methodology/Modeling emphasizes UML as Sketch

notes.arpanadhikari7.com.np

UML 1 and UML 2

- UML is being continually refined.
- UML 2 extends UML specifications to cover more aspects of development, including Agile.
- The goal was to restructure and refine UML so that usability, implementation, and adaptation are simplified.
- Some updates made from UML 1 to UML 2 are:

a. Greater integration between structural and behavior models.

b. Ability to define hierarchy and breakdown a software system into components and sub-components.

c. UML 2 raises the number of diagrams i.e. adds new types of diagrams.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Building Blocks of UML

1. Things: Things are the most important building blocks of UML
    a. Structural Things: The Structural things define the static part of the model. They represent physical and conceptual elements. Following are the brief descriptions of the structural things.
        a. Class
        b. Interface
        c. Collaborations
        d. Use Case
        e. Component
        f. Node
        g. Active Class
    b. Behavioral Things: A behavioral thing consists of the dynamic parts of UML models.
        a. Interaction
        b. State Machine

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

c. Grouping Things: Grouping things can be defined as a mechanism to group elements of a UML model together. There is only one grouping thing available:
    a. Package
d. Annotational Things: Annotational things can be defined as a mechanism to capture remarks, descriptions, and comments of UML model elements.
    a. Note

2. Relationships: . It shows how elements are associated with each other and this association describes the functionality of an application.

a. Association
    - Aggregation
    - Composition
b. Generalization
c. Realization
d. Dependency
e. Include and Extend

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

3. Diagrams
    1. Class diagram
    2. Object diagram
    3. Use case diagram
    4. Sequence diagram
    5. Collaboration diagram
    6. Activity diagram
    7. Statechart diagram
    8. Deployment diagram
    9. Component diagram

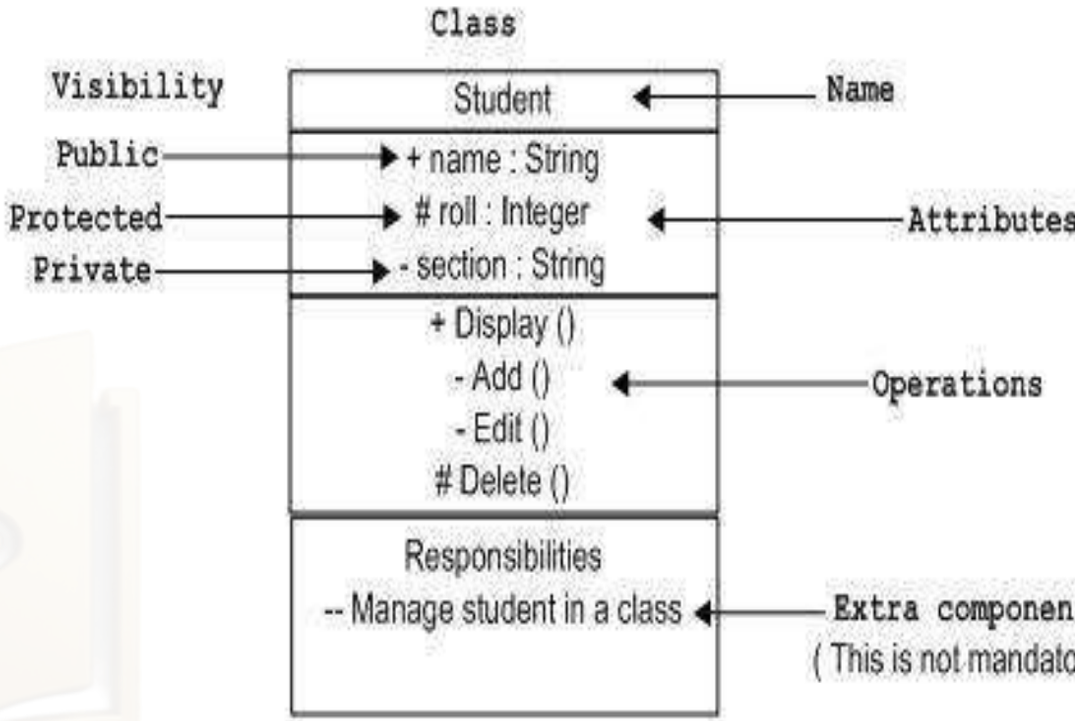


notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

UML Notations

1. Class Notation



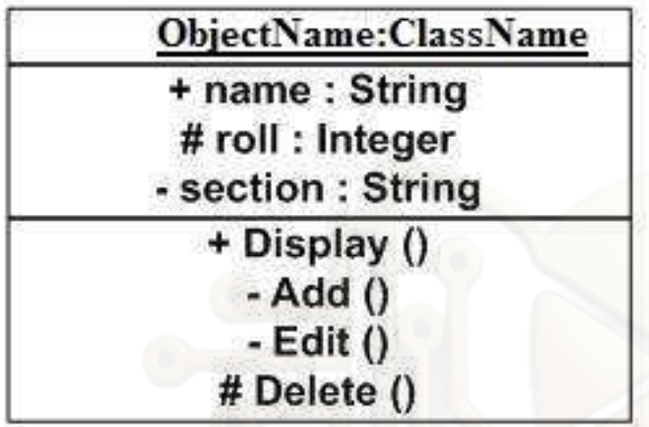
- The diagram is divided into four parts.
- The top section is used to name the class. The first letter of class name is capital.
- The second one is used to show the attributes of the class.
- The third section is used to describe the operations performed by the class.
- The fourth section is optional to show any additional components.
- \- symbol represents that attributes or operations are private.
- \+ symbol represents that attributes or operations are public.
- # symbol represents that attributes or operations are protected.
- ~ symbol represents package.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

2. Object Notation

- The object is represented in the same way as the class. The only difference is the **name which is underlined** as shown in the following figure.



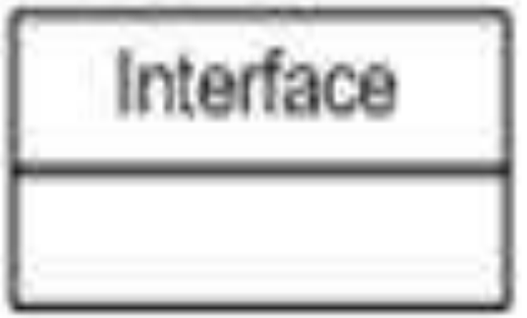
3. Interface Notation

- Interface is used to describe the functionality without implementation.
- Interface is just like a template where you define different functions, not the implementation.
- When a class implements the interface, it also implements the functionality as per requirement.

notes.arpanadhikari7.com.np

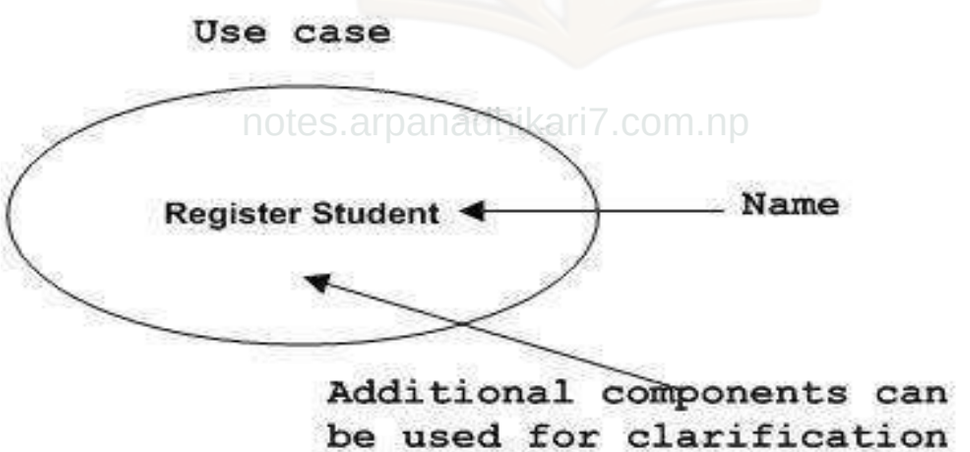
Available on notes.arpanadhikari7.com.np

- Interface is represented by a circle as shown in the following figure.



4. Use Case Notation

- Use case is used to capture high level functionalities of a system.
- It is represented as an eclipse with a name inside it.



notes.arpanadhikari7.com.np

5. Actor Notation

- An actor can be defined as some internal or external entity that interacts with the system.



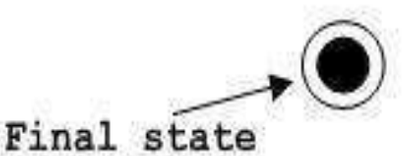
6. Initial State Notation

- Initial state is defined to show the start of a process.



7. Final State Notation

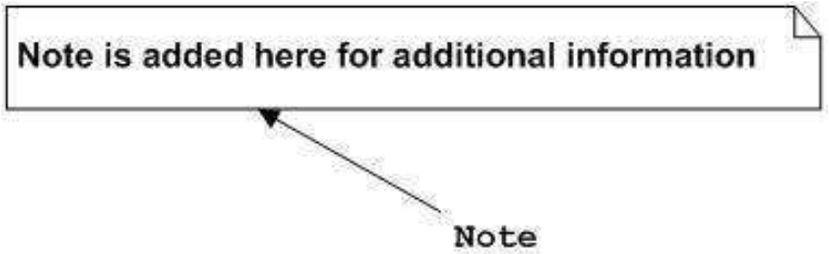
- Final state is used to show the end of a process.
- The usage of Final State Notation is to show the termination point of a process.



Available on notes.arpanadhikari7.com.np

8. Note Notation

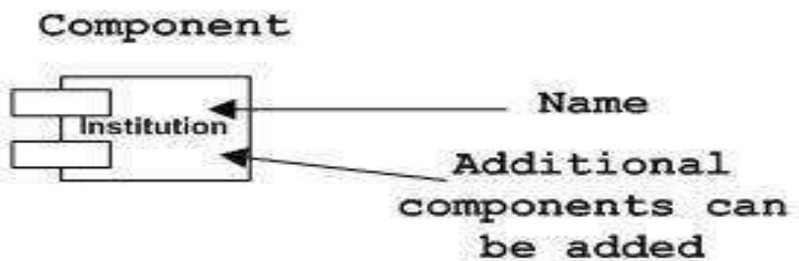
➢This notation is used to provide necessary information of a system.



9. Component Notation

➢Component represents a modular part of a system.

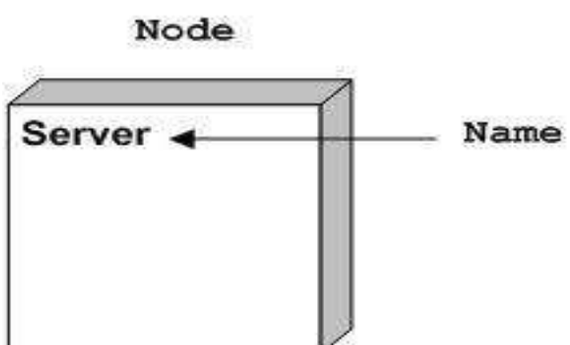
➢A component defines its behavior in terms of **provided and required interfaces.**



10. Node Notation

➢Node is used to represent the physical part of a system such as the server, network, etc.

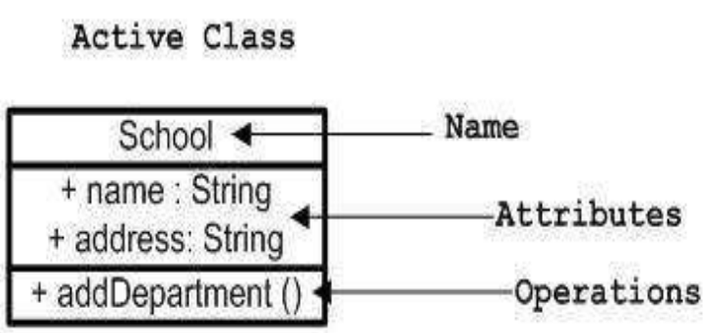
➢A node in UML is represented by a square box.



notes.arpanadhikari7.com.np

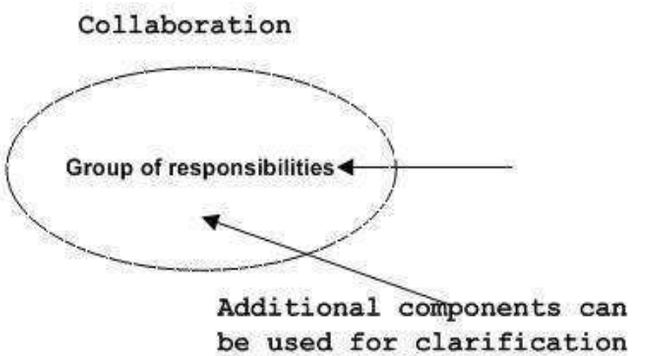
Available on notes.arpanadhikari7.com.np

11. Active Class Notation



- Active class looks similar to a class with a **solid border**.
- Active class is used to represent the concurrency in a system**.**
- An active class is a class whose instances are active objects.
- An active object is an object that owns a process or thread and can initiate control activity.

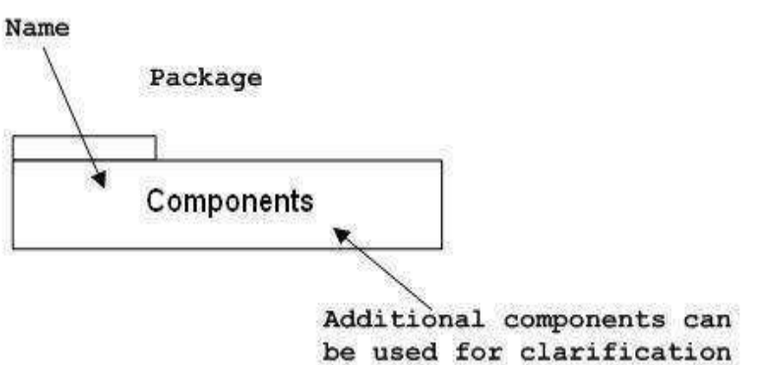
12. Collaboration Notation



- Collaboration represents responsibilities. Generally responsibilities are in a group.
- Collaboration is represented by a dotted eclipse.

notes.arpanadhikari7.com.np

13. Package Notation

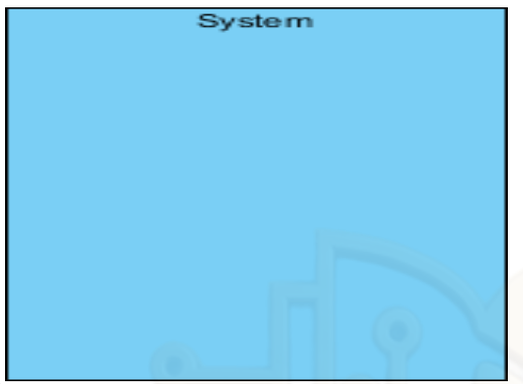


- Package is used to wrap the components of a system.

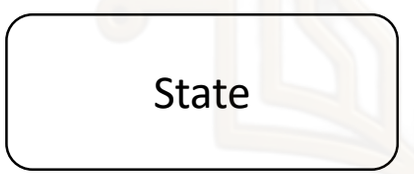
Available on notes.arpanadhikari7.com.np

14. System Notation

➢System in use-case diagram is represented by a rectangular box as shown below:



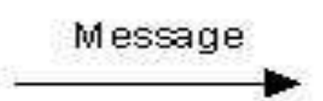
15. State Machine Notation



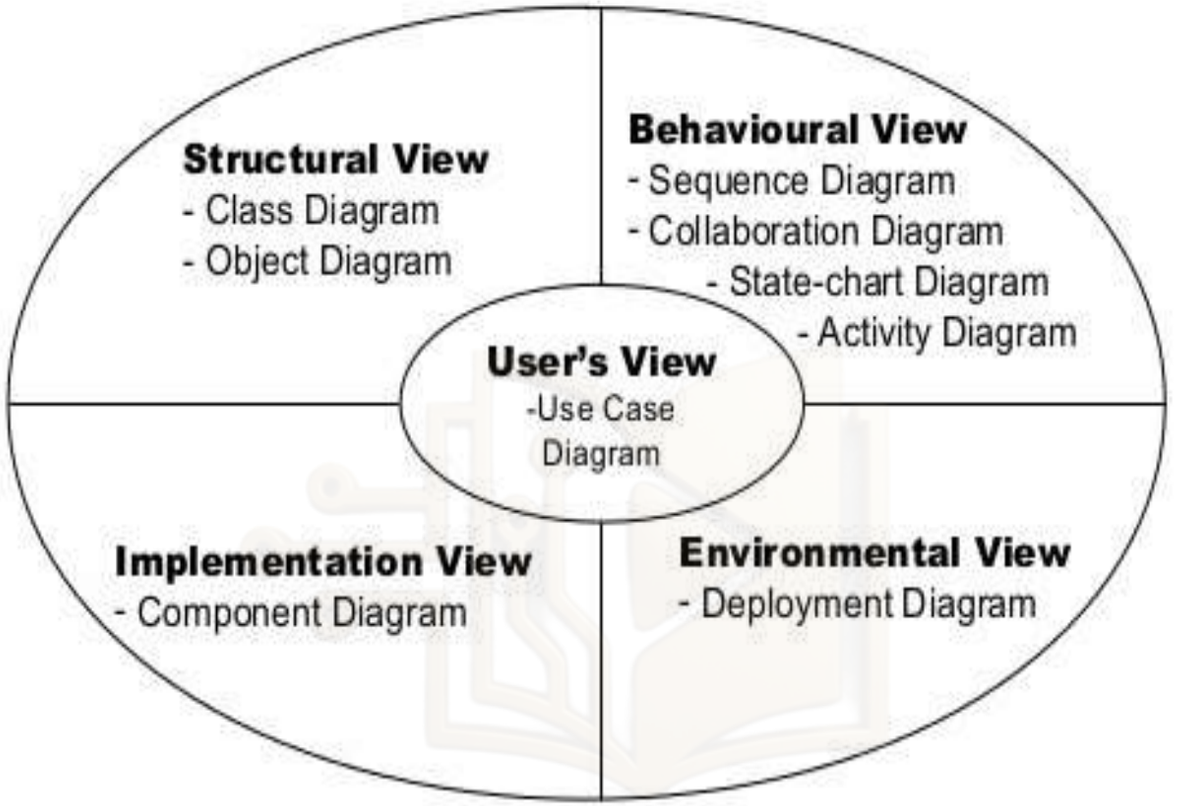
16. Message/Interaction

notes.arpanadhikari7.com.np

➢Interaction is defined as a behavior that consists of a group of messages exchanged among elements to accomplish a specific task.



Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Figure: Diagrams and Views Supported in UML1

Available on notes.arpanadhikari7.com.np

UML 2 Diagrams

Structural UML Diagram	Behavioral UML Diagram
Class Diagram	Activity Diagram
Object Diagram	Use Case Diagram
Component Diagram	Interaction Overview Diagram
Composite Structure Diagram	Sequence Diagram
Deployment Diagram	Communication(Collaboration) Diagram
Package Diagram	State Diagram
Profile Diagram	Timing Diagram

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Class Diagram

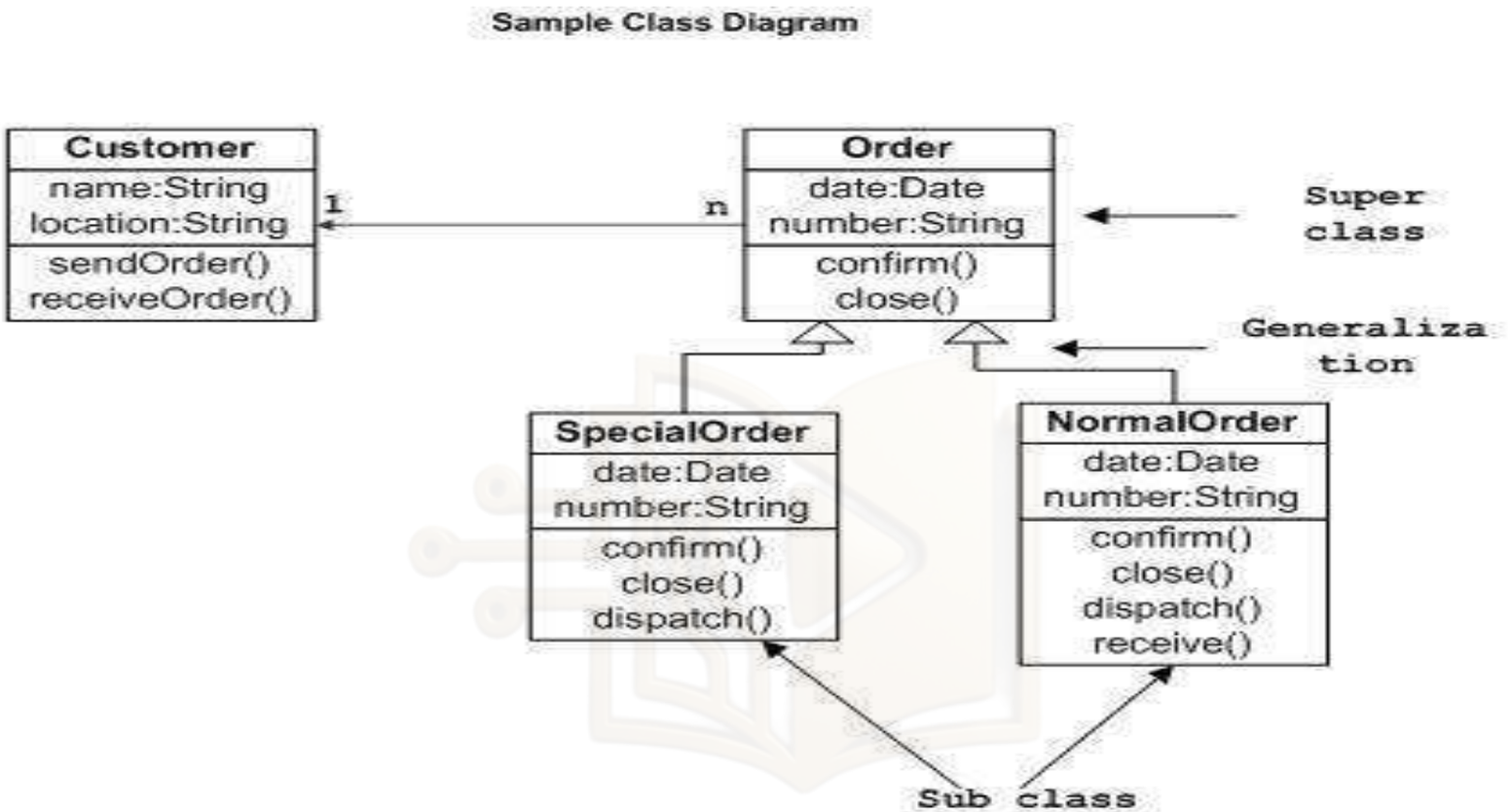
- Class diagrams are the main building block of any object-oriented solution.
- Class diagram is a static diagram. It represents the static view of an application.
- Class diagram is not only used for visualizing, describing, and documenting different aspects of a system but also for constructing executable code of the software application.
- The class diagrams are widely used in the modeling of object-oriented systems because they are the UML diagrams, which can be mapped directly with object-oriented languages.
- In most modeling tools, mainly a class has three parts.

a. Name at the top
b. Attributes in the middle
c. Operations or methods at the bottom.

- Different relationships between classes are shown by different types of arrow diagrams.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Figure: Class Diagram Example

Available on notes.arpanadhikari7.com.np

Object Diagram

- They show how a system will look like and relation of one object with the other at a given time.
- Object diagrams and class diagrams are closely related and use almost identical notation.
- Object diagrams are a set of objects and their relationship is just like class diagrams. They also represent the static view of the system.
- Object diagrams are derived from class diagrams so object diagrams are dependent upon class diagrams.
- An object diagram focuses on some particular set of objects and attributes, and the links between these instances.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Figure: Object Diagram Example

Available on notes.arpanadhikari7.com.np

Use Case Diagram

- Behavioral Diagram
- Use case diagrams model the functionality of a system using actors and use cases.
- A use case diagram is a graphic depiction of the interactions among the elements of a system.
- Use case diagrams are used to depict the context of the system to be built and the functionality provided by that system.
- They show what the outside world wants the system to do.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

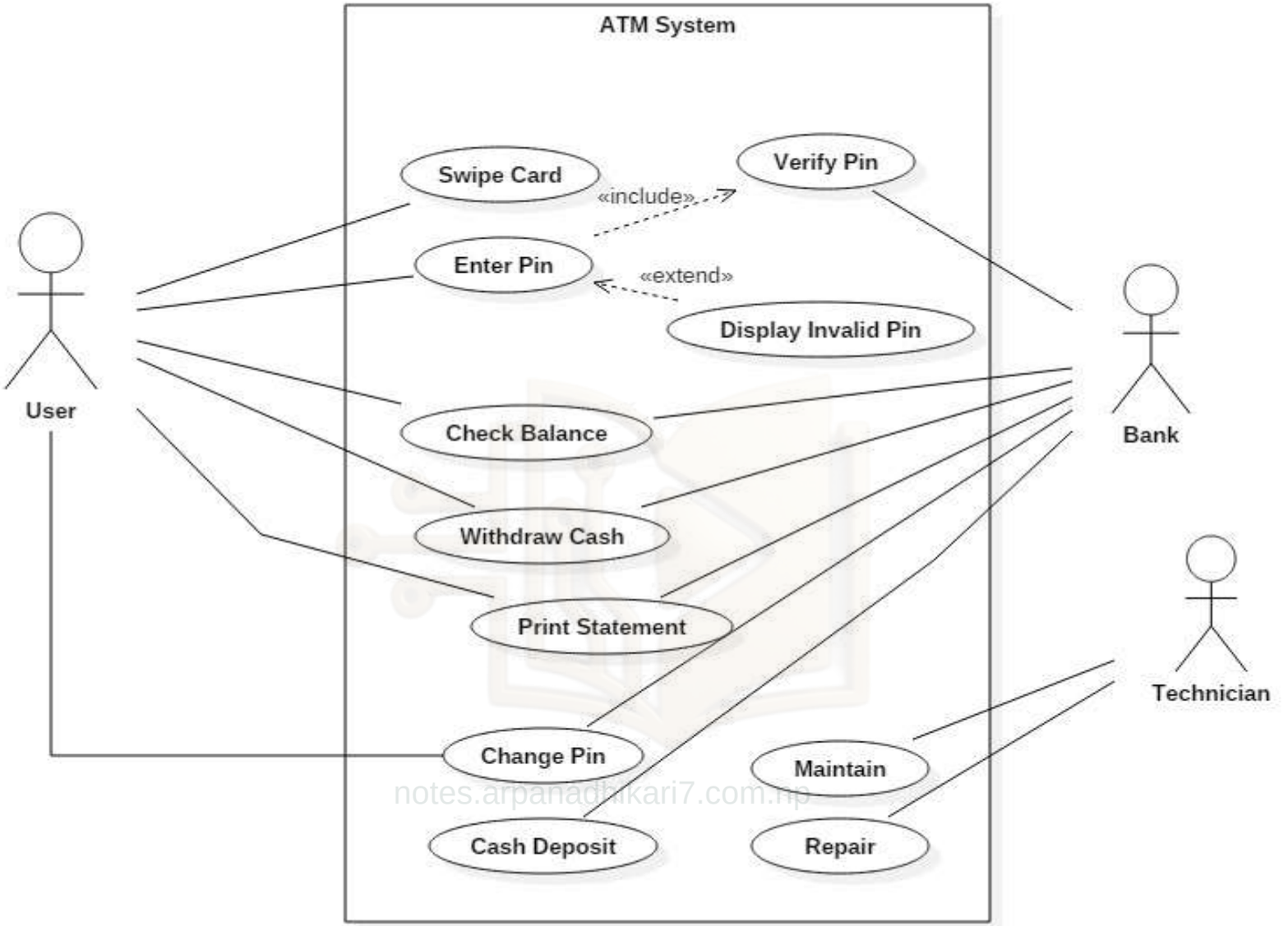


Figure: Use Case Diagram For ATM System

Available on notes.arpanadhikari7.com.np

Component Diagram

- A component diagram breaks down the actual system under development into various high levels of functionality.
- Component diagrams represent a set of components and their relationships. These components consist of classes, interfaces, or collaborations.
- During the design phase, software artifacts (classes, interfaces, etc.) of a system are arranged in different groups depending upon their relationship known as components.
- So it can be said component diagrams are used to visualize the implementation.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

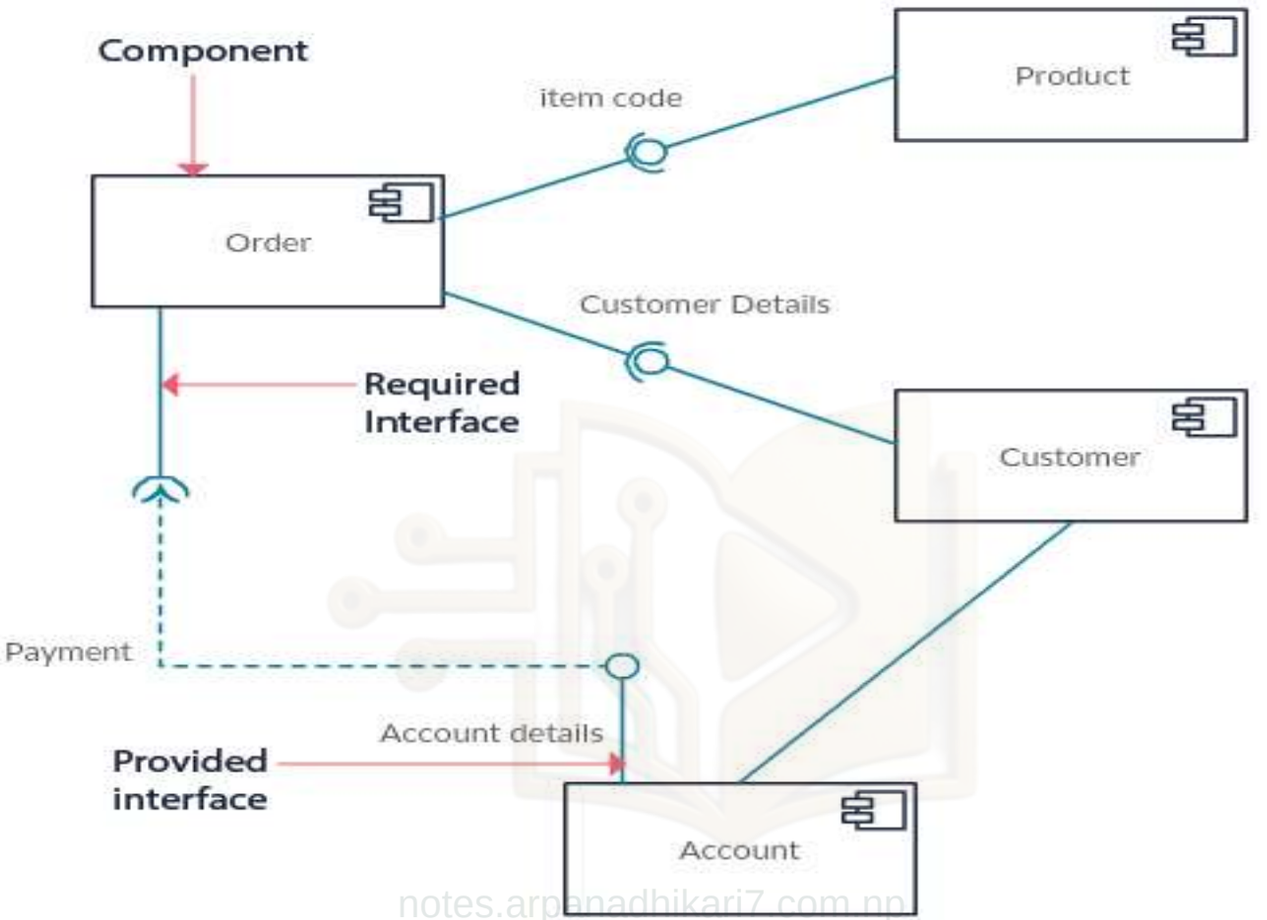
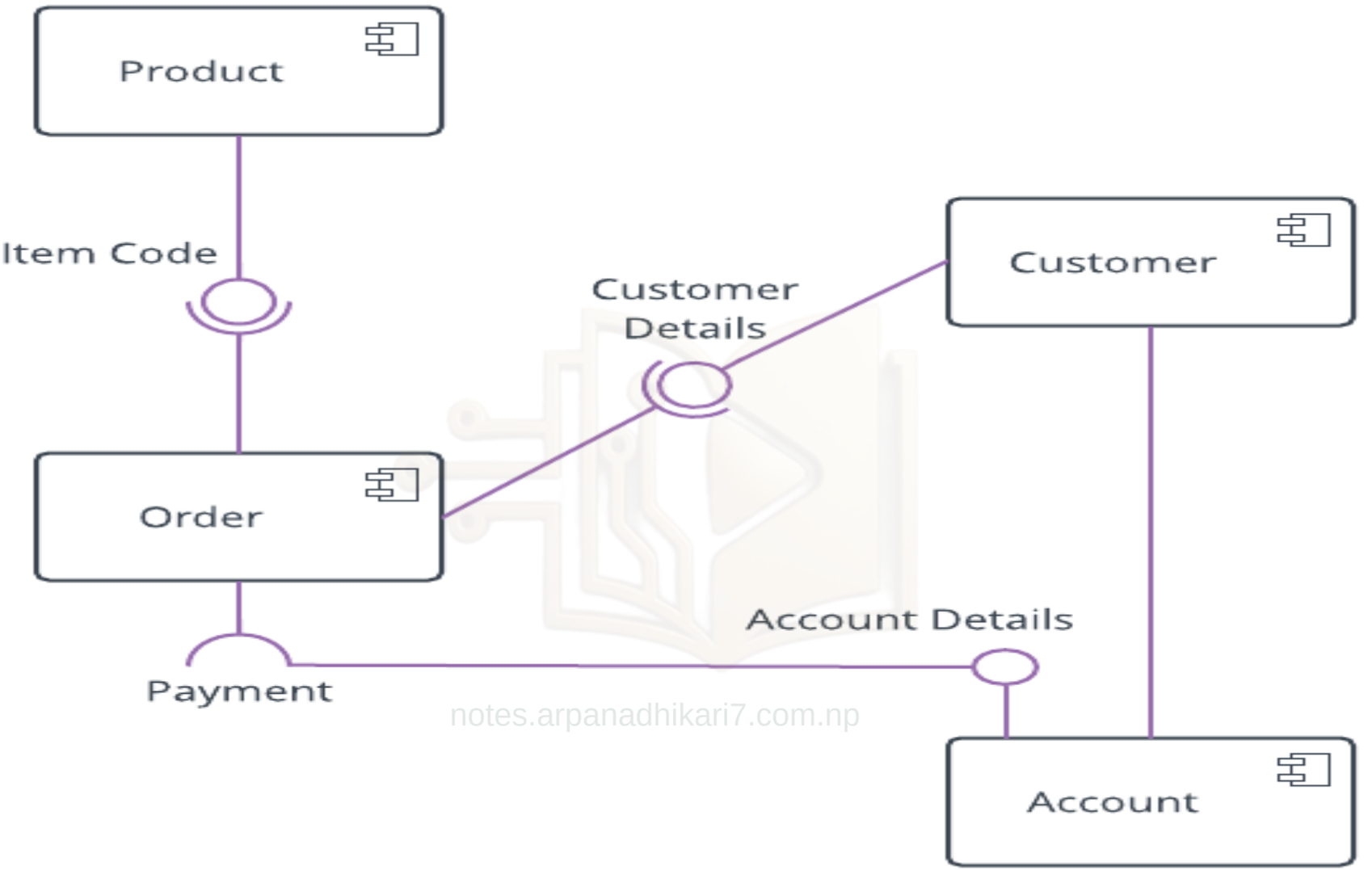


Figure: Component Diagram Example

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

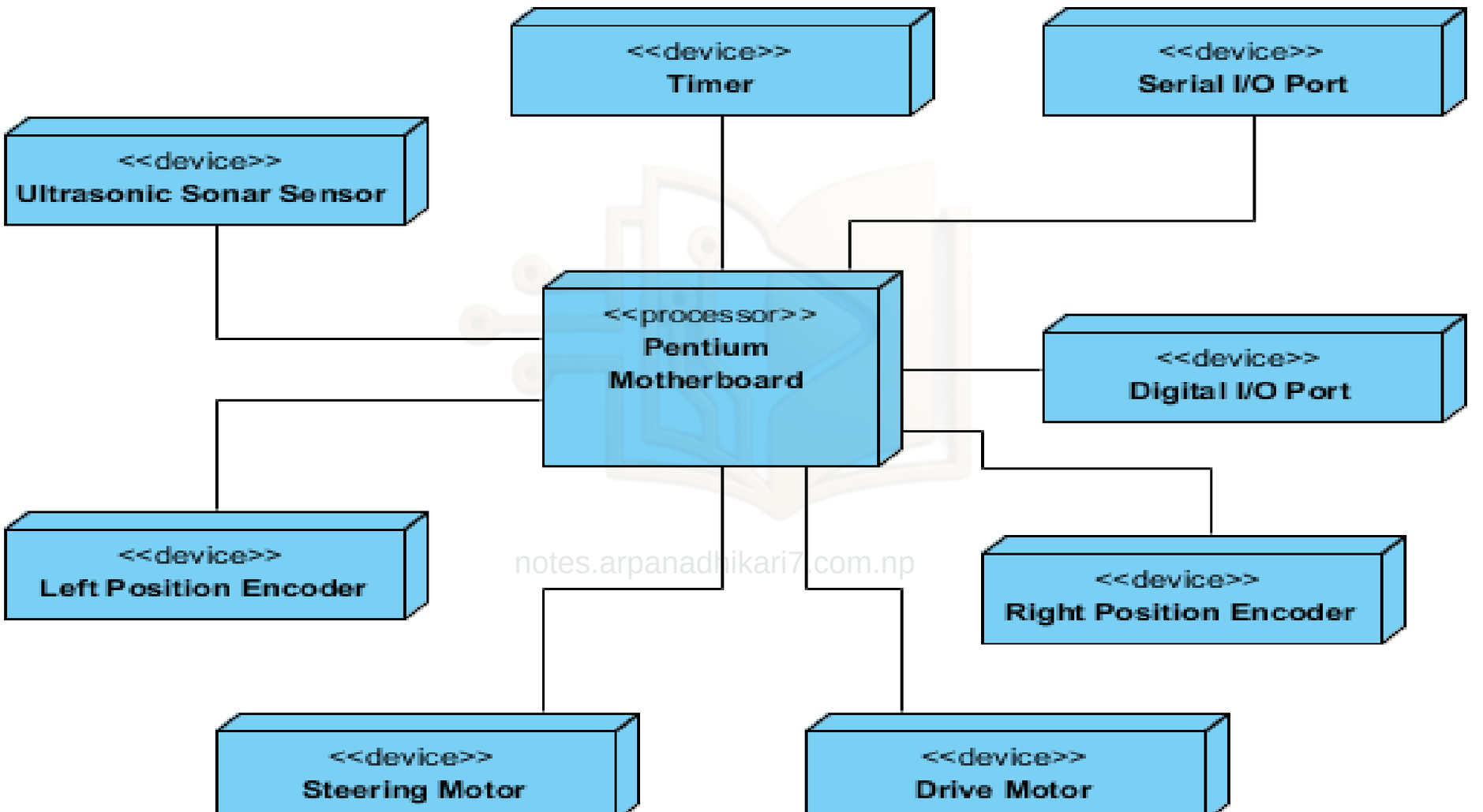
Deployment Diagram

- A deployment diagram is just a kind of class diagram, which focuses on a system's nodes.
- Deployment diagrams are used for describing the hardware components where software components are deployed.
- Graphically, a deployment diagram is a collection of vertices and arcs.
- Deployment diagrams are useful when your software solution is deployed across multiple machines with each having a unique configuration.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Deployment Diagram



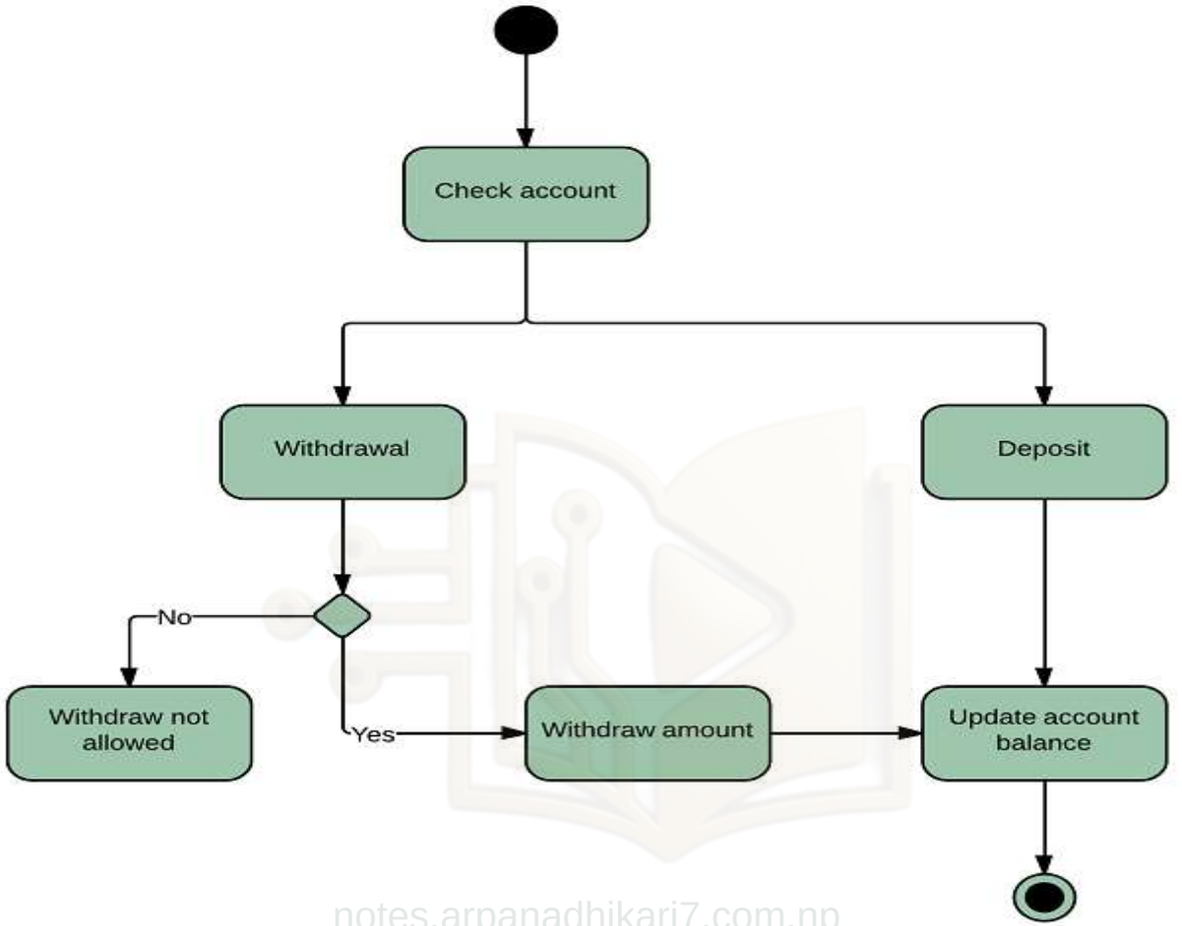
notes.arpanadhikari7.com.np

Activity Diagram

- It is a type of behavioral diagram that describes the flow of control in a system.
- Activities are nothing but the functions of a system.
- Numbers of activity diagrams are prepared to capture the entire flow in a system.
- The flow of control in activity diagram can be sequential, concurrent, or branched.
- This is prepared to have an idea of how the system will work when executed.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



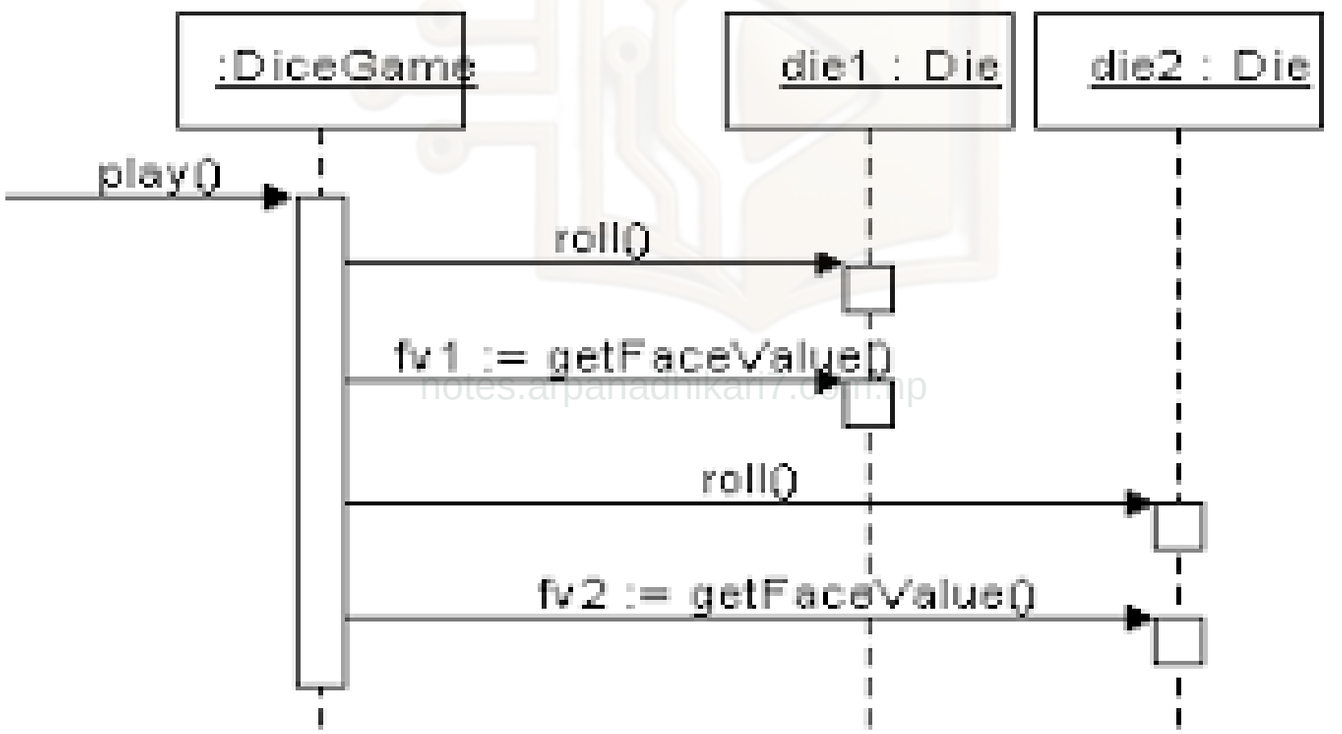
notes.arpanadhikari7.com.np

Figure: Activity Diagram Example

Available on notes.arpanadhikari7.com.np

Sequence Diagram

- Sequence Diagrams are interaction diagrams that detail how operations are carried out.
- A sequence diagram simply depicts interaction between objects in a sequential order i.e. the order in which these interactions take place.
- Sequence diagrams describe how and in what order the objects in a system function.



Available on notes.arpanadhikari7.com.np

Communication(Collaboration) Diagram

- Communication Diagram(also called collaboration diagram in UML 1.x) is a kind of interaction diagram which shows how objects interact using sequenced message in a free-form arrangement.
- In order to maintain the ordering of messages in such a free-form, messages are labeled with a chronological number and placed near the link the message is sent over.
- Communication diagrams show a lot of the same information as sequence diagrams

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

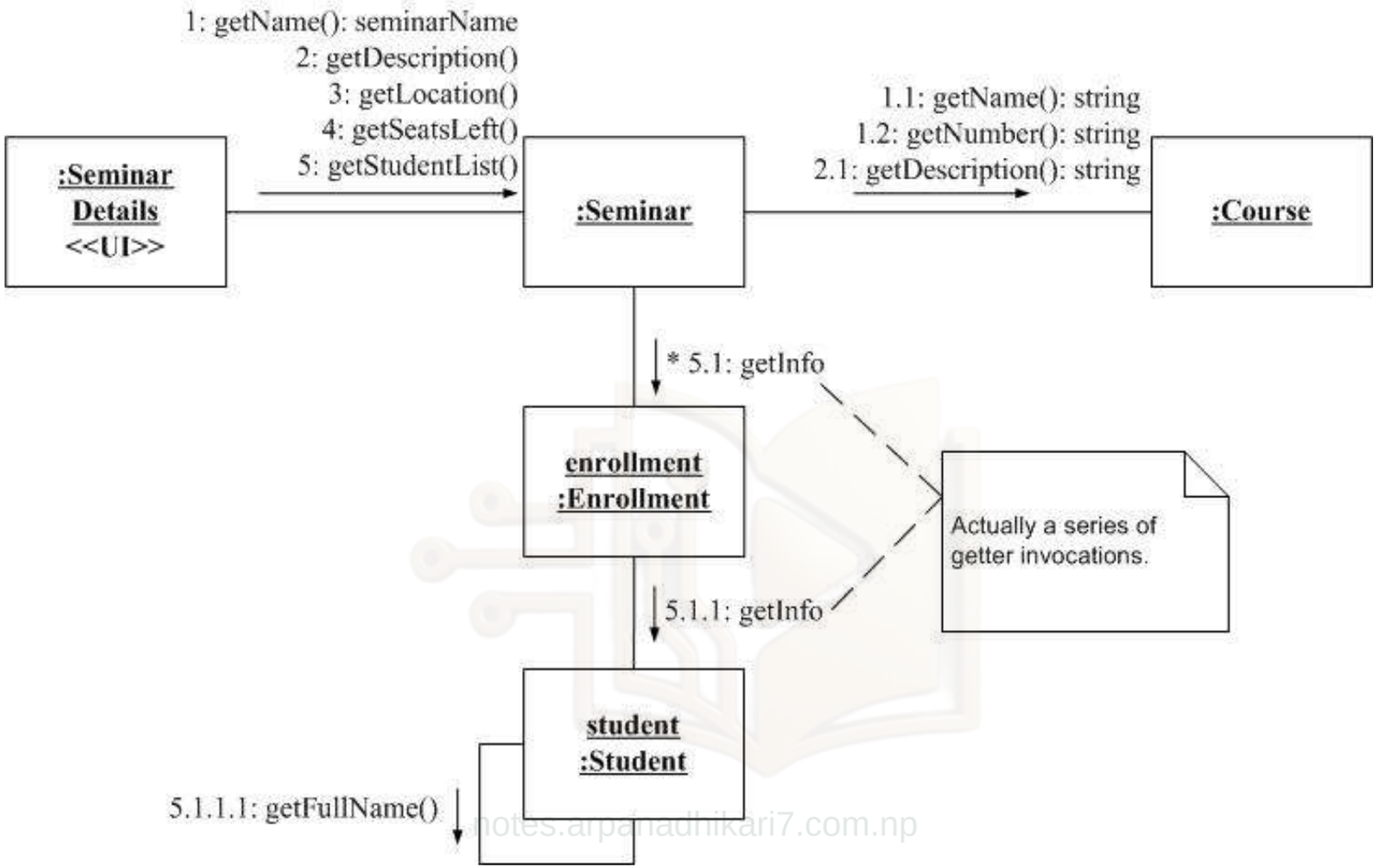
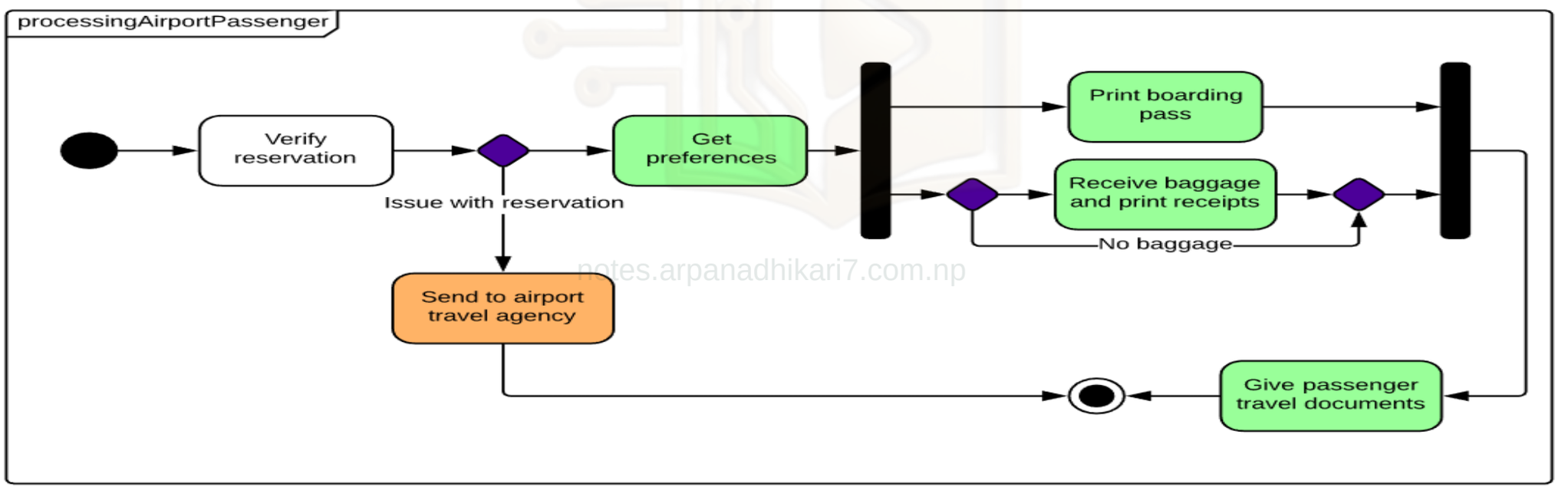


Figure: Communication Diagram Example

Available on notes.arpanadhikari7.com.np

State Diagram

- Also called as state machine diagram.
- A state machine is any device that stores the status of an object at a given time and can change status or cause other actions based on the input it receives.
- Each state diagram typically begins with a dark circle that indicates the initial state and ends with a bordered circle that denotes the final state



notes.arpanadhikari7.com.np

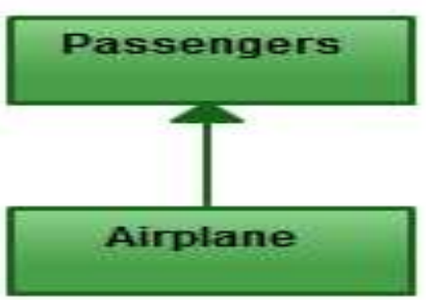
Relationships used in UML

1. Association

- An association relationship is a structural relationship between two model elements that shows that objects of one classifier (actor, use case, class, interface, node, or component) connect and can navigate to objects of another classifier.
- In bidirectional relationships, an association connects two classifiers, the primary (supplier) and secondary (client) and is called binary associations. A binary association is represented by a line. An association that connects n number of classifiers is known as n-nary association.
- Association is represented by a solid line. Line can also feature multiplicity rules (eg. one-to-one, one-to-many, many-to-many) for the relationship.



notes.arpanadhikari7.com.np



Directed Association

Available on notes.arpanadhikari7.com.np

2. Reflexive Association

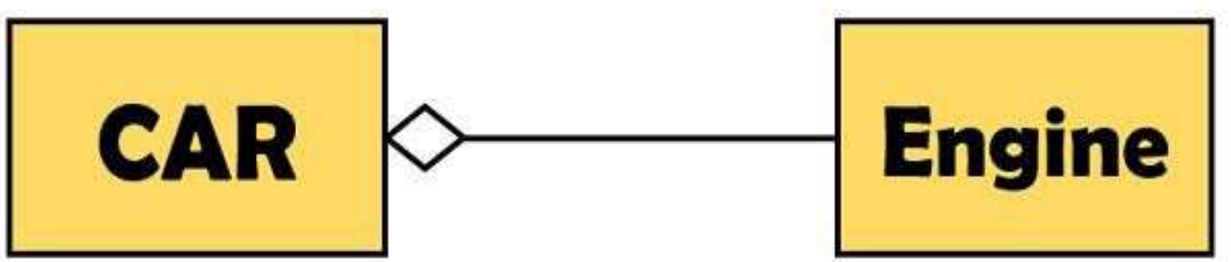
- A reflexive association is an association from a class to itself.
- This relationship is shown when a class may have multiple functions or responsibilities.



notes.arpanadhikari7.com.np

3. Aggregation

- It is a type of association relationship where a child can exist independently of the parent.
- Example: Class (parent) and Student (child). Delete the Class and the Students still exist.
- The aggregation is a type of association which describes a **“has a”** type of relationship between the objects.
- Aggregation is represented as a solid line with a hollow diamond head.
- It is a weak type of association.
- The deletion of assembly doesn't affect its parts.

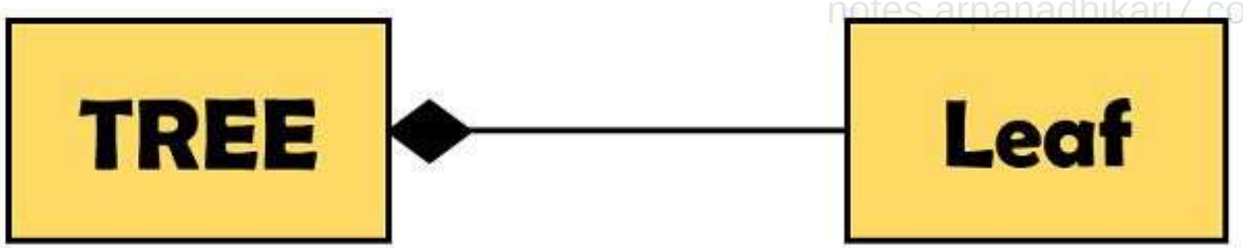


Available on notes.arpanadhikari7.com.np

4. Composition

- It is a type of association relationship where a child cannot exist independently of the parent.
- Example: House (parent) and Room (child). Rooms don't exist separate to a House.
- The composition is a type of association which describes a **"part of"** type of relationship between the objects.
- Composition is represented as a solid line with a solid diamond head.
- It is a strong type of association.
- The deletion of assembly doesn't affect its parts. For example, a leaf is a part of a tree, if the tree is destroyed, then leaves are must be destroyed.

notes.arpanadhikari7.com.np





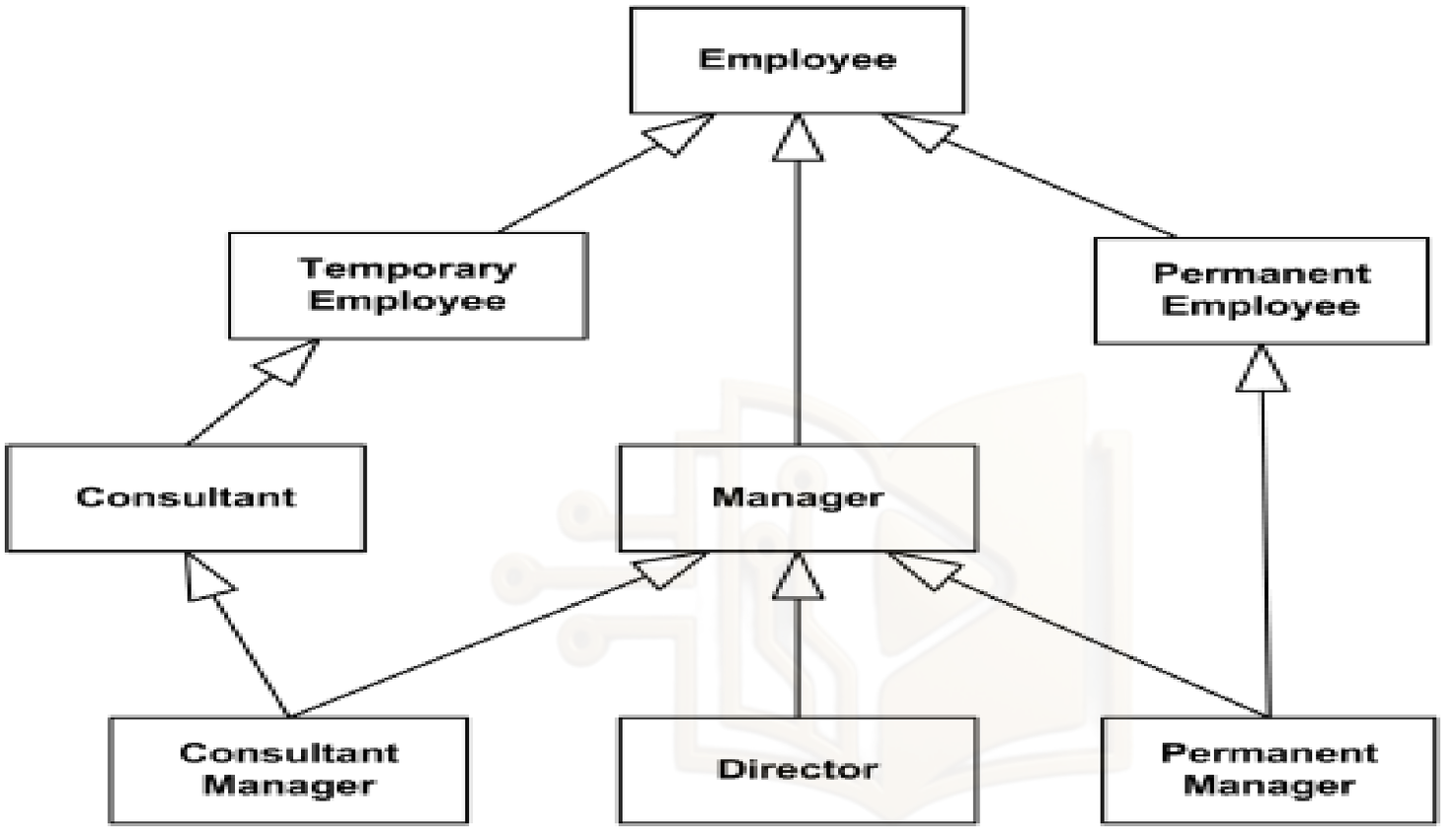
Available on notes.arpanadhikari7.com.np

5. Generalization or Specialization

- It is a relationship in which one model element (the child) is based on another model element (the parent).
- Generalization relationships are used in class, component, deployment, and use-case diagrams to indicate that the child receives properties, attributes, operations that are defined in the parent.
- Generalization relationship is also informally called "**Is A**" relationship.
- Generalization relationships do not have names.
- A generalization is shown as a line with a hollow triangle as an arrowhead between the symbols. The arrowhead points towards the parent entity or element.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



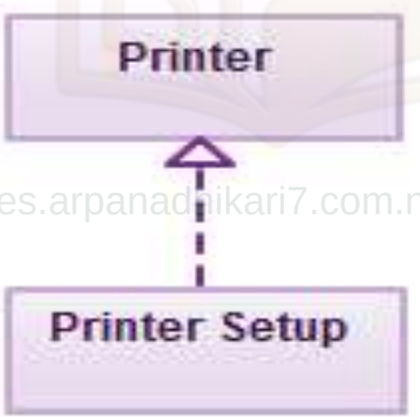
notes.arpanadhikari7.com.np

Figure: Generalization Relationship in UML

Available on notes.arpanadhikari7.com.np

6. Realization

- A realization relationship is a relationship between two model elements, in which one model element (the client) realizes the behavior that the other model element (the supplier) specifies.
- Several clients can realize the behavior of a single supplier.
- It denotes the implementation of the functionality defined in one class by another class. In other words, you can understand this as the relationship between the interface and the implementing class.



notes.arpanadhikari7.com.np

- To show the relationship in UML, a broken line with an unfilled solid arrowhead is drawn from the class that defines the functionality to the class that implements the function.
- In above example the printer setup interface or class is being implemented by the printer class.

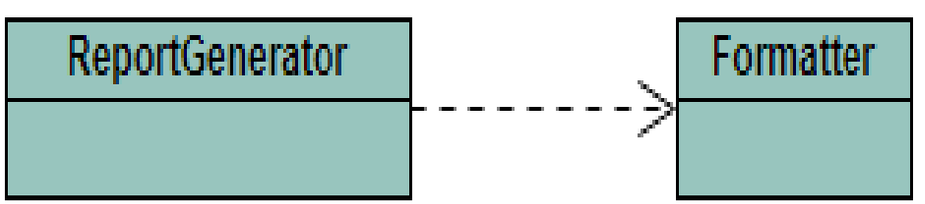
notes.arpanadhikari7.com.np

7. Dependency

- A dependency relationship is a relationship in which one element, the client, uses or depends on another element, the supplier.
- So changes made in one element(on which the other is depending) may affect the element which is dependent on the other.
- If a class A "uses" class B, then one or more of the following statements generally hold true:

- Class B is used as the type of a local variable in one or more methods of class A.
- Class B is used as the type of parameter for one or more methods of class A.
- Class B is used as the return type for one or more methods of class A.
- One or more methods of class A invoke one or more methods of class B.

notes.arpanadhikari7.com.np



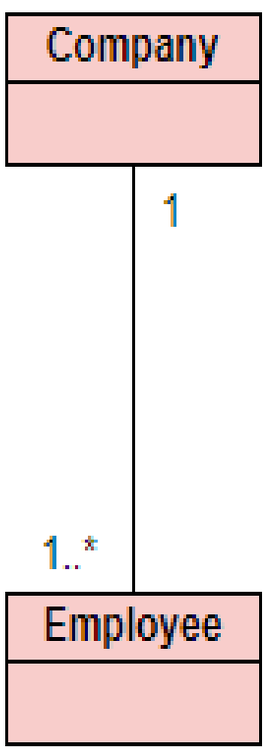
Available on notes.arpanadhikari7.com.np

8. Include and Extend Relationship

Already Discussed

9. Multiplicity

- Multiplicity in UML allows to specify cardinality - i.e. number of elements associated.



Multiplicities examples:

1	Exactly one, no more and no less
0..1	Zero or one
*	Many
0..*	Zero or many
1..*	One or many

notes.arpanadhikari7.com.np